Ecosystems, Tipping Points, and Catastrophes

Joshua S. Weitz[1, 2, *]

[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*

(Dated: November 20, 2020)

I. MODELING COMPLEXITY: AN ENABLING VIEW

Ecosystem dynamics arise from the interactions of heterogeneous agents, a combination of biotic and abiotic factors, and often strong influences of stochasticity. Such a combination of factors would tend to lead the quantitative study of particular ecosystems towards larger and larger models, with ever more interactions, feedback, parameters, and so on. Yet as described in the accompanying chapter, features of ecosystem dynamics can often transcend the particular details. What then is the right balance to strike? If made too simplistic, models become no more than metaphors or worse, objects meant to be admired, not falsified. If too complex, then it becomes difficult, if not impossible, to identify the salient factors that govern key phenomena other than in a statistical insight. This series of lab guides has attempted to strike a balance by contextualizing and motivating the biology (and physics, when necessary) of model choices so as to connect dynamical models with mechanisms. Nonetheless, this kind of work, when adopted at ecosystem scales poses certain challenges.

Precisely so, this final computational laboratory departs from the rule in the spirit of the adage: "the exception that proves the rule". The exception is precisely because models of ecosystems do, in fact, tend to include many many components, including population abundances, intra-species variants, nutrient inputs, patches/habitats for colonization, and other physical drivers that coupled to ecology. They are the closest manifestation of the warning by Rosenblueth and Weiner on model-building: "The best material model of a cat is another, or preferably the same, cat." Yet what has given life to these models are a few simple ideas, e.g.: small changes in conditions can lead to large changes in outcomes; states – when entered – can persist for far longer and under more extreme conditions than it took to enter them; certain types of feedback can lead to runaway, positive feedback events that can lead ultimately to catastrophe.

In essence, the point of the models in this laboratory is to identify the dynamical ingredients that lead to the emergence of chaos, catastrophes, and alternative stable states in community and ecosystems. Because we are focusing on ingredients, each of the model sections will be 'small', and build in complexity, motivated by recent models connected to observational datasets that are only slightly more involved. One final note. The analysis of time series data is critical to any analysis. Prior laboratories have emphasized a time-series perspective, which is also relevant to the concepts here. The models here are broken into three components.

First, a simplified model of logistic growth is presented using discrete time steps - representative of seasons or growing periods. This model is related to some of the earliest models in the class with a major exception: even this simple model can exhibit chaos. Second, the laboratory switches back to a continuous frame, again turning to the logistic model, with an exception: namely that the model assumes that the population reshapes its environment. By reshaping the environment, it is possible that a population can increase its growth rate, but in certain cases this feedback ends not in a steady state but a catastrophe. Finally, the last model treats a particular model of a looming catastrophe: the Earth System. In doing so, we focus on a particular low-dimensional model of climate, shaped by microbes, but where in some sense their actions are implicit. The model reveals, as you will see, that if we think about the Earth as a dynamical system, then like neurons and cardiac cells, it is possible that rapid influx of organic carbon can lead to excitations and even long trajectories far from the equilibrium state.

II. SMALL DIFFERENCES, BIG EFFECTS

How do small variations in initial conditions or parameters affect the long-term dynamics of populations, communities, and ecosystems? For linear systems, the answer is trivial: small variations have small effects. However, for nonlinear systems, the answer depends on the feedback of the system itself. One might expect that the variation

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

could range from small to large, depending on the system. What is more interesting is that for nonlinear dynamical systems, the answer to this question can be extreme. In some cases, any variation, however small, will eventually lead to a complete loss of predictability/correspondence between two otherwise identical systems. This is the butterfly effect in a nutshell, that a seemingly imperceptible change in the system conditions can lead to a completely different outcome. Let's see how this works in perhaps the simplest model that exhibits this phenomena – the discrete logistic model.

A. Logistic growth vs. logistic maps

The very earliest models in this laboratory guide and accompanying textbook focused on problems related to bacterial reproduction. Successive divisions could be described as a continuous process, as

$$\frac{\mathrm{dN}}{\mathrm{dt}} = rN \tag{1}$$

where r is the growth rate of the population, such that $N(t) = N_0 e^{rt}$ given an initial abundance of N_0 at time $t = 0$. Notably, given that $N(t) = 2N_0$ when $e^{rt} = 2$, then the "doubling" time can be identified as

$$\tau = \frac{\log 2}{r}. \tag{2}$$

Yet, bacteria (or any other species for that matter) cannot grow indefinitely. Instead, there will be limits. One way to impose limits is to denote the growth of the population via a logistic model such that

$$\frac{\mathrm{dN}}{\mathrm{dt}} = rN\left(1 - \frac{N}{K}\right). \tag{3}$$

Here, as long as $r > 0$ and $N_0 > 0$, there can only be one possible outcome: the system will converge rapidly to $N = K$, on the time scale of order $1/r$ (i.e., similar to the doubling time).

Yet, not all processes are best described by continuous dynamics, particularly those with growing seasons or other staged/structured periods of growth and decay. Given the nature of doubling, the following discrete growth model would seem equivalent to the continuous model:

$$N_{t+1} = RN_t \tag{4}$$

where R denotes a discrete growth factor and N_t denotes the population at time t. The solution is $N_t = N_0 R^t$, where $\log R$ is the discrete exponential growth factor. As such, as long as $R > 1$, the system will grow in population. But nothing grows forever. Hence, first extend this model to include a logistic feedback term, i.e.,

$$N_{t+1} = RN_t\left(1 - \frac{N_t}{K}\right) \tag{5}$$

To do so, enter in the following model code given $R = 1.5$ or develop one on your own. Either way, overlay the results of two sets of initial conditions – one relatively large and the other relatively low – to visualize the dynamics as follows. Once you have a working code, try out the next challenge problem.

Challenge Problem: Finding the Equilibria

Utilize the discrete logistic model and find the equilibria for the cases $R = 1.2$, 1.5 and 1.8. Visualize the dynamics to show that simulations agree with the analytical predictions.

```
import numpy as np
import matplotlib.pyplot as plt
#parameters
pars={}
pars['R']=1.5
pars['K']=100
pars['tmax']=20
pars['dt']=1
#Simulation
t = np.arange(0,pars['tmax']+pars['dt'],pars['dt'])
N = np.zeros(np.shape(t))
N[0]=1
for i in range(len(t)-1):
    N[i+1]=pars['R']*N[i]*(1-N[i]/pars['K'])
#Visualization
plt.plot(t,N,c='k',marker='o',markerfacecolor='w')
plt.xlabel('Time, t',fontsize=20)
plt.ylabel('Population, N',fontsize=20)
```

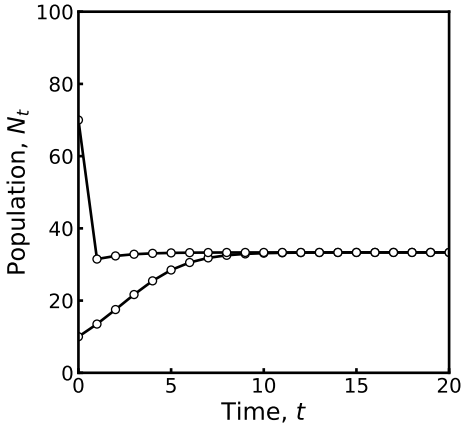
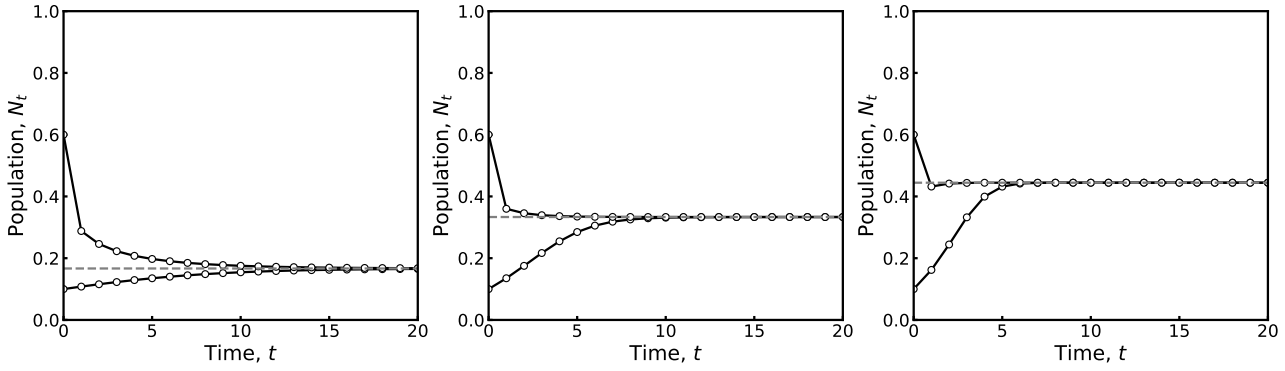


FIG. 1: Logistic model with $R = 1.5$ and $K = 100$.

Solutions to Challenge Problem: Finding the Equilibria
The equilibrium point for the dynamics satisfies $N(t+1) = N(t) = N^*$. This is true for $N^* = 0$ and for $N^* = 1 - R^{-1}$. Hence, the predicted dynamics should be 1/6, 1/3 and 4/9 for the values of $R = 1.2$, 1.5, and 1.8, respectively.



This logistic model can be normalized by rewriting the model in terms of the fractional abundance $x_t = N_t/K$, such that

$$x_{t+1} = Rx_t\left(1 - x_t\right). \tag{6}$$

In this model the fractional abundance is contrained such that $0 \leq x_t \leq 1$. This model will be the basis for the next section of analysis.

B. Bifurcations and oscillations in logistic maps

The discrete logistic model denotes a *map* between the current state of the system x_t and the next state of the system x_{t+1}. Formally, this is $f : X_t \to X_{t+1}$. To get started, use the following code to visualize the map. As you will see, the system will have 0 population when initialized at either $x = 0$ or $x = 1$, and will have a maximal population level at the next time period when initialized at $x = 0.5$.

This map can also be used as the basis for understanding long-term dynamics. Consider the following procedure. First, start with the current state, x_t. Then, use the map to identify the next state, x_{t+1}. Then, if one uses the map a second time, it is possible to identify the next-next state, i.e., x_{t+2}. Visually this amounts to 'cob-webbing', re-mapping the y-value back to the x-value and then mapping again and again. Depending on the value of R, the system may converge to a single equilibrium state, x^*, to multiple values (denoting oscillations in the dynamics), or to a series that is completely unpredictable over the long term (chaos!!!). The next challenge problem is your chance

```
# Parameters
pars={}
pars['R']=3.2
x0 = np.arange(0,1.001,0.001)
# The model
disc_logistic = lambda x: pars['R']*x*(1-x)
#Plot the map
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
plt.plot(x0,disc_logistic(x0),c='k',linewidth=3)
```

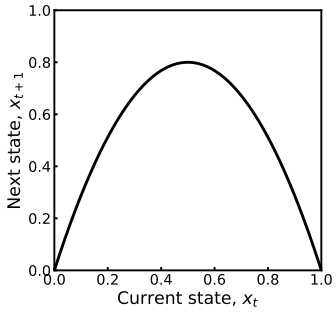
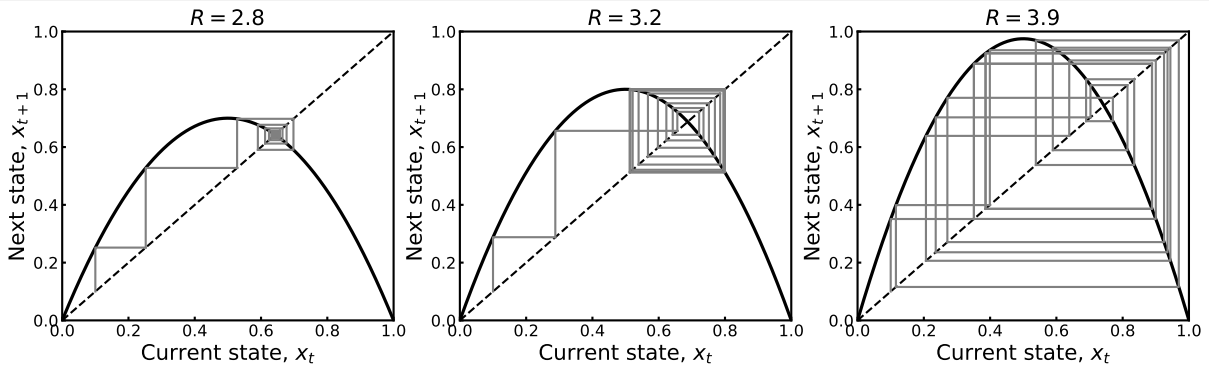


FIG. 2: Discrete logistic model map with $R = 3.2$

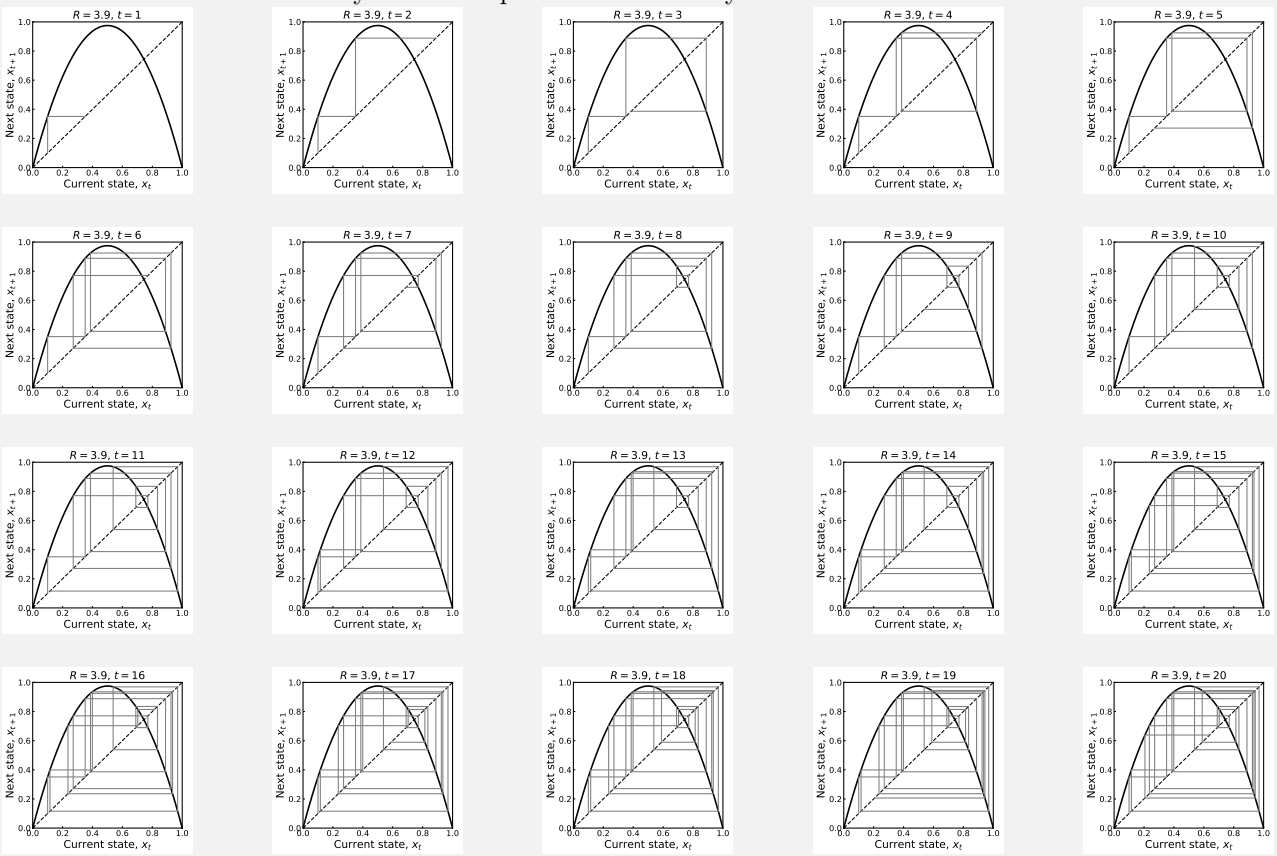
to write a cob-webbing code, and then explore the dynamics in the long-term. Are long-term dynamics predictable (or not) despite the fact that the near-term dynamics are completely predictable.

Challenge Problem: Cobwebbing Dynamics

Write code that simulates the discrete logistic model and overlays the results directly on the map. Use a duration of 20 time points (think of them as growing seasons). Do this for three values of R, i.e., $R = 2.8$, $R = 3.2$, and $R = 3.9$. If the code is working, it should look like:



Hint: The key insight here is to think of cobwebs as combinations of two-line segments. These two-line segments form brackets connecting three sets of points: (i) (x_t, x_t); (ii) (x_t, x_{t+1}), and (iii) (x_{t+1}, x_{t+1}). Repeated again and again should yield a visualization of the dynamics that can be overlaid with the map (as shown above). This hint can be seen, in action, in the sequence of snapshots, a form of cobweb animation, in the case of $R = 3.9$ – this animation is beyond the scope of this laboratory.



Solutions to Challenge Problem: Cob-webbing Dynamics
One approach to this problem is to break this into three components. First, plot the map. Second, overlay the one-to-one line. Third, simulate the model step by step and plot a sequence of two-line segments (as noted in the hint). The solutions also include labeling commands. The answers are as follows:

```
# Parameters
pars={}
pars['Rvals']=[2.8, 3.2, 3.9]

rNum=len(pars['Rvals'])
fig, axes = plt.subplots(1,3,figsize=(5*rNum+2*(rNum-1),5))
for i in range(rNum):
    ax = axes[i]

    #Conditions
    pars['R']=pars['Rvals'][i]
    pars['tmax']=25
    x0=0.1
    # The model
    disc_logistic = lambda x: pars['R']*x*(1-x)

    # the map
    xrange =np.arange(0,1.001,0.001)
    ax.plot(xrange,disc_logistic(xrange),'k',linewidth=3)
    ax.plot(xrange,xrange,'k',linewidth=2,linestyle='--')

    #Simulate
    x = np.zeros(pars['tmax']+1)
    x[0]=x0
    for n in range(pars['tmax']):
        x[n+1]=disc_logistic(x[n])
        ax.plot([x[n], x[n], x[n+1]],[x[n], x[n+1], x[n+1]],
             c=[0.5,0.5,0.5], linewidth=2)
    #Annotate
    ax.set_ylim(0,1)
    ax.set_xlim(0,1)
    ax.set_xlabel(r'Current state, $x_t$',fontsize=20)
    ax.set_ylabel(r'Next state, $x_{t+1}$',fontsize=20)
    ax.set_title(r'$R={R}$'.format(R=pars['R']),fontsize=20)
```

Note that this code can be adapted into an animation by pausing before plotting again in the figure. Give the command `\%matplotlib qt` in the command window, then at the end of the loop say `plt.pause(0.01)` and `plt.show()`. This allows the code to overlay new lines at a rate slow enough to see. Alternatively, use the `matplotlib.animation` package. The code in the loop would be encoded in an update function. The resulting animation will come complete with a play button. This is an exercise for the very interested reader.

C. Evidence of long-term divergence of trajectories despite short-term predictability

How much will two systems differ in the long-term even when they start off at nearly the same initial condition? The answer to this question requires significant elaboration. But, in practice, it is possible to measure divergence for the discrete logistic model (and indeed for other systems). Consider two trajectories, x_n and x'_n which are initialized at the initial conditions, x_0 and $x_0 \pm \delta_0$. Formally, the divergence for the discrete logistic model is:

$$\delta x(n) \equiv |x_n - x'_n| \tag{7}$$

The discrete logistic model has an unusual property, for certain values of R, namely that trajectories that start very close to each other become increasingly divergent at an exponential rate! This can be seen visually by contrasting the dynamics when $R = 2.8$ vs. that of $R = 3.9$. Using the code below, generate a pair of time series separated

by a small deviation, e.g., 0.0001. Then, calculate the divergence and note whether it increases or decreases. This code is the basis for further explorations of the fact that the divergence doesn't just increase or decrease, but does so exponentially (as is evident by the nearly-linear nature of the divergence with time when plotted on semilogy axes). The code below provides a solid basis for the exercise:

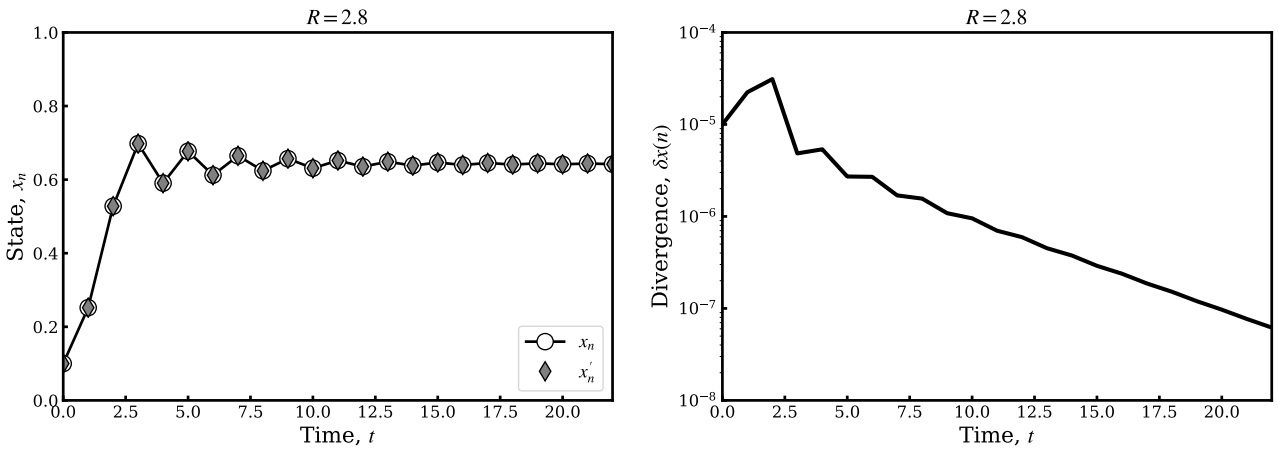
```
# Initialize the parameters
pars={}
pars['R']=2.8
pars['tmax']=25
pars['x0']= [0.1, 0.10001]

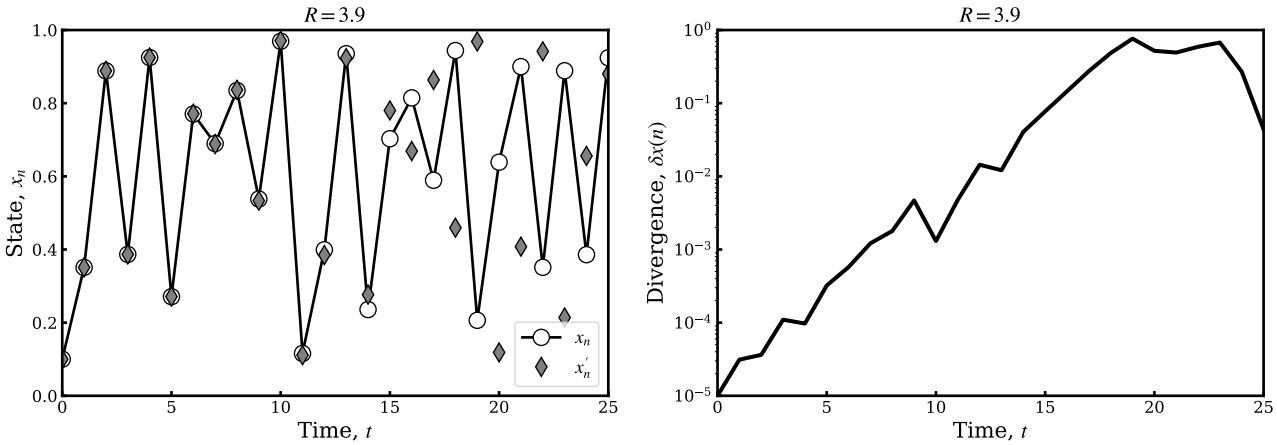
# Simulations
x=np.zeros((pars['tmax']+1,len(pars['x0'])))
for i in range(len(pars['x0'])):
    x[0,i]=pars['x0'][i]
    for j in range(pars['tmax']):
        x[j+1,i]=pars['R']*x[j,i]*(1-x[j,i])
t=np.arange(pars['tmax']+1)

# Divergence
deltax = np.abs(x[:,0]-x[:,1])

# Plot Divergence
fig, axes = plt.subplots(1,2,figsize=(16,5))
ax = axes[1]
ax.plot(t,deltax,'k',linewidth=3)
ax.set_yscale('log')
ax.set_xlabel('Time, $t$',fontsize=20)
ax.set_ylabel(r'Divergence, $\delta x(n)$',fontsize=20)
# Plot the states
ax = axes[0]
ax.plot(t,x[:,0],c='k',linewidth=2,
        marker='o',
        markerfacecolor='w',
        markersize=12)
ax.plot(t,x[:,1],linestyle='',
        marker='d',markerfacecolor=[0.5,0.5,0.5],markeredgecolor='k',
        markersize=10)
ax.legend([r'$x_n$',r"$x_n'$"],fontsize=14,loc='lower right')
ax.set_title(r'$R={R}$'.format(R=pars['R']),fontsize=20)
ax.set_xlabel(r'Time, $t$',fontsize=20)
ax.set_ylabel(r'State, $x_n$',fontsize=20)
```

If you decide to also plot the trajectories along with the divergence, you will find the following:





As is evident, trajectories for the case of $R = 2.8$ converge rapidly, indeed it is not even possible to see the difference in the trajectory plot. In contrast, the trajectory divergence reaches nearly the limit of the system range after 20 iterations in the case of $R = 3.9$. This exponential increase in divergence is a hallmark of chaos; a topic for post-laboratory work.

III. EXPLOSIVE GROWTH AND POPULATION CATASTROPHES

A. On feedback between populations and their environment

Populations have a limited capacity to growth. The limitations are myriad: space, nutrients, stress, and other populations. Yet populations also have the ability to reshape their environment. In certain realms of ecology, such ideas are often framed as niche construction. The term 'niche' refers to the idea, sensu G. Evelny Hutchinson, that there are certain combinations of biotic and abiotic factors that enable an organism to persist. These fundamental constraints may be greater than the realized constraints due to competition and other ecological feedbacks. Niche construction suggests that organismal behavior, when compounded at the population scale, can lead to feedback with the envrionment, reshaping it so that the environment is in fact more conducive to population growth. In layperson's terms, one can think of a beaver dam. In a crude sense, agriculture is also a form of transforming an evironment to expand the viability of a particular environment that has another consequence: increasing the potential for increases in population density. But for many microbes, their interactions and transformation of carbon and nutrients can fundamentally transform the Earth system – like the oxygenation of Earth. Hence, this next section is meant to explore what happens when a population is limited by its environment (as in the logistic growth model) but with a twist: the population growth reshapes the environment, potentially increasing the capacity for growth. So far so good. But as the model will show, such feedback can have unexpected and even catastrophic outcomes.

B. Preamble – logistic growth model

The logistic model is one of the simplest in the study of living systems. The logistic model describes the initial exponential growth of a population at a rate r that eventually reaches a saturating carrying capacity, K, i.e.,:.

$$\frac{\mathrm{dN}}{\mathrm{dt}} = rN\left(1 - \frac{N}{K}\right) \tag{8}$$

The solution to this model depends on r, K and N_0, the population abundance at $t = 0$. Given that this is one of the last laboratories in this series, it is worth asking yourself: can you start with an entirely blank file, simulate the logistic model, and confirm that your answers are indeed correct – without checking with a friend or Google? For an experienced coder this is a straightforward task. But these labs are intended to be used by scientists and students, including those with limiting coding experience. Hence, to the extent possible, try to do this next challenge problem without notes and see how far you can go.

Challenge Problem: Continuous Logistic Model From Scratch

Write code that simulates the logistic model, over a period of 0 to 100 days, assuming a population that begins at a level $N_0 = 50$, given a maximum growth rate $r = 0.1$ and carrying capacity $K = 10^5$. **Important note:** The solution to this problem has already been documented at the very outset of these laboratories. But that was a long time ago. In many courses, the same question is asked as a pre- and post-exam to evaluate learning (and retention). Consider this your own benchmark.

Solutions to Challenge Problem: Continuous Logistic Model from Scratch:
The following code represents one way to solve this problem, but not the only way. Here, the solutions separate the function from the script for later use and re-use. First the script:

```
# Model specification
t = np.arange(0,200,0.1)
pars={}
pars['r']=0.1
pars['K']=10**5
N0=50

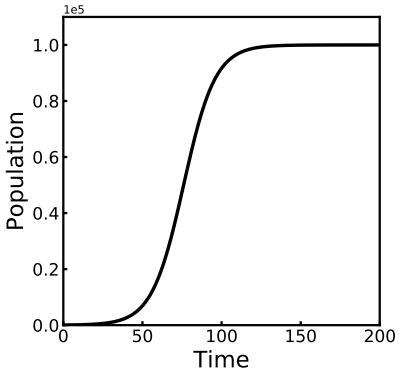
#Simulate
y= integrate.odeint(logistic_model,N0,t,args=(pars,))

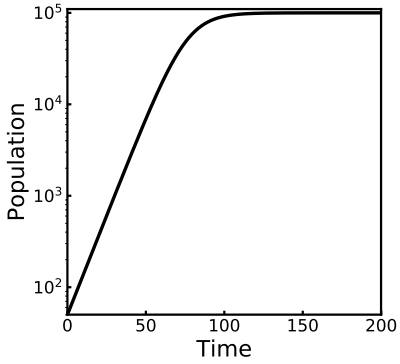
# Plot and label
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
plt.plot(t,y,'k',linewidth=3)
plt.xlabel('Time',fontsize=20)
plt.ylabel('Population',fontsize=20)
plt.xlim(min(t),max(t))
plt.ylim(0,1.1*pars['K'])
```

and then the function:

```
def logistic_model(N,t,pars):
    """
    Logistic model dynamics
    dNdt = rN*(1-N/K)
    """
    dNdt = pars['r']*N*(1-N/pars['K'])
    return dNdt
```

This code yields the following figure (on left). And with one simple change – shifting the `plot` command to `semilogy` it can also yield the figure on the right where the initial exponential growth is self-evident given the straight line dynamics on a log-scaled y axis.





C. Malthus-Condorcet model

The logistic model represents the impact of *negative* density dependence on the growth of a particular population. But what if the density were to have a stimulatory effect, rather than an inhibitory effect, on growth. Such a model can be considered by generalizing the logistic model as follows:

$$\frac{\mathrm{dN(t)}}{\mathrm{dt}} = \rho N(t)\left(K(t) - N(t)\right) \tag{9}$$

$$\frac{\mathrm{dK(t)}}{\mathrm{dt}} = c\frac{\mathrm{dN(t)}}{\mathrm{dt}} \tag{10}$$

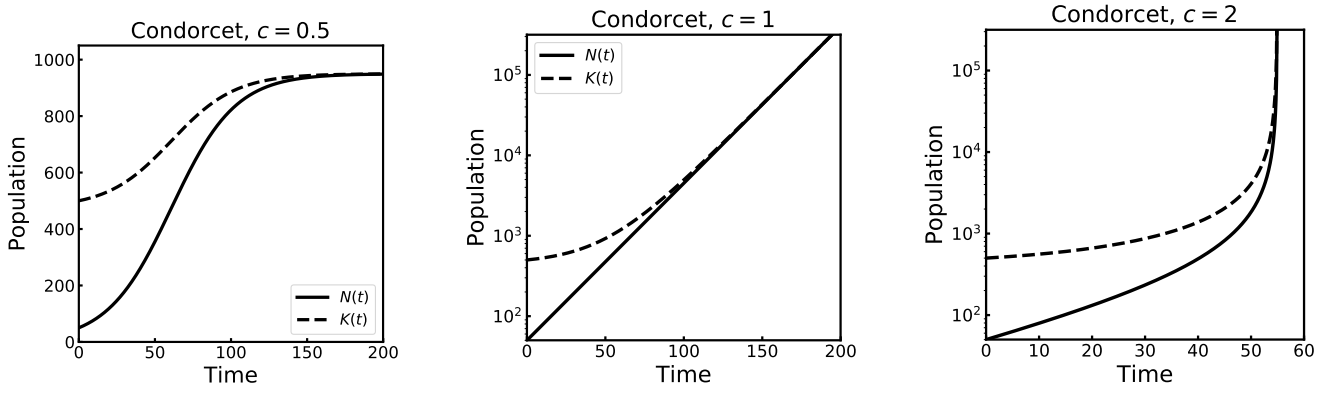
The simulation code for this model is easily adapted from the logistic growth model, i.e.,

```
def condorcet_model(y,t,pars):
    """
    Malthus-Condorcet model dynamics
    dNdt = rho*N*(K-N)
    dKdt = c*dNdt
    """
    # initialize
    N=y[0]
    K=y[1]

    #Find derivatives
    dNdt = pars['rho']*N*(K-N)
    dKdt = pars['c']*dNdt
    return np.array([dNdt, dKdt])
```

We will refer to this as the Malthus-Condorcet model; see main text for more deatils.

The key concept in the Malthus-Condorcet model is that the carrying capacity increases at a rate equal to that of the rate of change of the population modified by the Condorcet parameter, c. It is also important to note that in this formulation the Malthus parameter ρ has a slightly different interpretation. For example, when $N \ll K$, then the growth rate is exponential with a rate of ρK. This model can be mapped to the logistic model when $c = 0$. However, three regimes are relevant for some form of positive feedback $c > 0$: (i) $c < 1$; (ii) $c = 1$; (iii) $c > 1$. In a fundamental sense, c, indicates whether or not the rate of expanding resources/capacity of a system scales slower then, equal to, or faster than the population. Without explaining the rationale for these regimes yet, it is worthwhile to explore the dynamics using the following parameters and initial conditions $\rho = 10^{-4}$, $K_0 = 500$, $N_0 = 50$ albeit given $c = 0.5$, $c = 1$, and $c = 2$.



These three models all start with the same initial conditions. If K had been constant, then the maximum growth rate would be $\rho K_0 = 0.05$ setting a time scale of approximately 20. But the carrying capacity is not constant. Instead, in each case the carrying capacity increases, leading to a qualitative change in dynamics.

For the case $c = 0.5$, the resulting population dynamics appear to be logistic, with a twist. The carrying capacity nearly doubles from 500 to ≈ 950. As the population approaches this saturating level, the difference between K and N decreases, and because $c < 1$, the carrying capacity increases less than does the population. Over time the population equals that of the carrying capacity and both $\dot{N}$ and $\dot{K}$ equal 0.

Next, for the case $c = 1$, the system grows exponentially after a transient – note the use of log-scaled population axes in this plot. The main text explains this in more detail, nonetheless - the key idea is that every bit of increase

in population leads to a concomitant increase in the carrying capacity. Hence, the population can never reach it, and in fact, the population remains as far from the carrying capacity as it was at the very start. Note that for the case at long times the dynamics converge to $N(t) \sim Ae^{\tilde{r}t}$, where A is a constant. Because $c = 1$, then N and K must have the same exponential growth rates, $\tilde{r}$ but could differ by a factor – C, such that $K(t) = Ae^{\tilde{r}t} + C$. In this case, the factor is $C \equiv (K(0) - N(0)$ and the effective exponential growth rate is $\rho\,(K(0) - N(0))$.

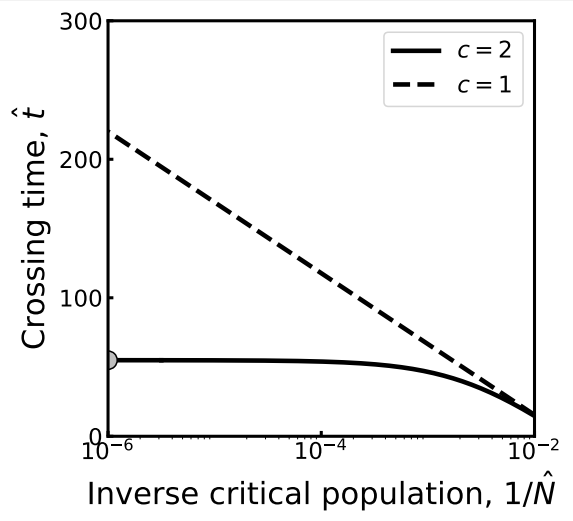
Finally, for the case of $c = 2$, the system does something seemingly unexpected. It grows faster than exponential! The faster than exponential growth can be identified by the positive curvature of $N(t)$ on log-scaled population axes. Yet, in running the code, there is another kind of problem: any standard differential equation solver fails to actually solve the equation. That seems strange. Strange enough that it's worth sharing the error.

```
ODEintWarning: Excess work done on this call (perhaps wrong Dfun type).
  Run with full_output = 1 to get quantitative information.
  warnings.warn(warning_msg, ODEintWarning)
```

The error does not arise because of some error in coding, instead it arises because of a fundamental change in the dynamics: the population blows-up, explodes to infinity, encounters a singularity, or any one of similar euphemisms. What is striking, is that in contrast to the exponential growth model in which populations go to infinity – rapidly to be sure – but in an infinite amount of time, the Malthus-Condorcet model reaches infinity in a finite time when $c > 1$! This seems impossible, and of course in the real world, the system would not reach an infinite population. But it would spike up in a seeming singularity, increasing super-exponentially, to a time in the not so distant future – a 'doomsday' scenario caused by the ability of the system to continually expand its capacity for growth. Trying to identify that finite time underlies the next challenge.

'Challenge Problem: Positive feedback and population explosions'

Modify and extend the following code to identify the finite time singularity in a logistic model with positive feedback, i.e., the Malthus-Condorcet model. In doing so, try to explain the choice to plot the time, $\hat{t}$ for $N(t)$ to cross a critical value $\hat{N}$ on inverse and log-scaled population axes. A working code should yield the following image:



Notice that in the code below, the actual dynamical system must also be coded up as a function. In addition, you will need to code up a modification to the standard numerical integration to check for crossing events. *Hint: for detecting crossings in Python* consider using the `np.argmax` function which returns the first index that satisfies an input condition. Another option would be to use `scipy.integrate.solve_ivp` rather than the usual `scipy.integrate.odeint`. This new function would allow the user to write event detection code to terminate the calculation before any errors occur.

```
# Crossing points c=2
Ncrit = np.logspace(2,6,1000)
t=np.arange(0,300,0.1)
pars['rho']=0.0001
pars['c']=2
pars['K0']=500
pars['N0']=50
y = integrate.odeint(...,args=(pars,),rtol=...)

#Find crossing points
tcrit = np.zeros(np.shape(Ncrit))
for i in range(len(Ncrit)):
    tmpi=np.argmax(...)
    tcrit[i]=...

plt.plot(...,linewidth=3)
```

Solutions to Challenge Problem: Positive feedback and population explosions: The following code includes a script that simulates the model with fixed parameters and systematically finds the time in which the population, $N(t)$ crosses a series of thresholds. The code then plots $1/\hat{N}$ – the thresholds – on the x-axis against the crossing time, $\hat{t}$ on the y-axis. Note that for exponential growth $\hat{t} \sim log(\hat{N})$, so that there is no finite time convergence to an infinite population for $c = 1$ in the Malthus-Condorcet model. The solution has one additional part, a crossing code, also included here. First, the main code:

```
# Crossing points c=2
Ncrit2 = np.logspace(2,5.5,1000)
t=np.arange(0,200,0.0001)
pars['rho']=0.0001
pars['c']=2
pars['K0']=500
pars['N0']=50
y = integrate.odeint(condorcet_model,np.array([pars['N0'],pars['K0']]),t,
                     args=(pars,),rtol=10**-8)

# Find crossing points c=2
tcrit2 = np.zeros(np.shape(Ncrit2))
for i in range(len(Ncrit2)):
    tmpi=np.argmax(y[:,0]>Ncrit2[i]) #find first point that exceeds value
    tcrit2[i]=t[tmpi]


# Plot
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
plt.plot(1/Ncrit2,tcrit2,'k',linewidth=3)
ax.set_xscale('log')

# Crossing points c=1
Ncrit = np.logspace(2,6,1000)
t=np.arange(0,400,0.0001)
pars['rho']=0.0001
pars['c']=1
pars['K0']=500
pars['N0']=50
y = integrate.odeint(condorcet_model, np.array([pars['N0'],pars['K0']]),t,
                     args=(pars,),rtol=10**-8)

#Find crossing points
tcrit = np.zeros(np.shape(Ncrit))
for i in range(len(Ncrit)):
    tmpi=np.argmax(y[:,0]>Ncrit[i])
    tcrit[i]=t[tmpi]

plt.plot(1/Ncrit,tcrit,'k',linestyle='--',linewidth=3)
```

Here, as a bonus, are the plot labeling commands:

```
# Singular point
plt.plot([10**-6, np.min(1/Ncrit2)],[54.9306, np.max(tcrit2)],'k',linewidth=3)
plt.plot(10**-6, 54.9306,marker='o',
         markeredgecolor='k',markerfacecolor=[0.75,0.75,0.75],markersize=12)

plt.legend([r'$c=2$', r'$c=1$'],fontsize=15,loc='upper right')
plt.xlabel(r'Inverse critical population, $1/\hat{N}$',fontsize=20)
plt.ylabel('Crossing time, $\hat{t}$',fontsize=20)
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=15,direction='in',width=2)
plt.xticks([10**-6,10**-4,10**-2])
plt.yticks([0,100,200,300])
```

IV. SMALL MODELS OF A BIG CLIMATE

The final case study in this laboratory considers the problem of how and when a complex Earth system can change states. But in doing so, there are some concessions to make. Full-scale models of the Earth system are far beyond the present scope. These models include three-dimensional coupling of hydrodynamics, chemistry, biology, and the integration of processes across massive spatiotemporal scales. There are good reasons to develop and contribute to such collaborative efforts – whether at MIT, NASA, or elsewhere. Indeed, these models can make a scope of predictions that a simple model cannot make. Yet, a simple model has another advantage: it has the potential to provide insight and perhaps even a narrative – with 'characters', 'story', and 'arc' – to understand how perturbations in a nonlinear system can, given certain feedback, drive a system from a seemingly long-term resilient state to somewhere very different. This last lab is, in that sense, a model of a model – but one that many more may need to understand if we are to avert the outcomes they foretell. The model was developed by Daniel Rothman of MIT, it represents a culmination of over a decade of work to try and understand the mechanisms that led to mass extinctions in the past and, further, what those events can teach us to forestall such catastrophic extinctions in the present.

A. Baseline carbon cycle model

This model of the Earth's carbon cycle focuses on the "shallow" surface of the ocean, i.e., the photic zone. Carbon concentrations in this shallow zone reflect a combination of fluxes, including input from photosynthesis in which inorganic carbon (CO_2) is 'fixed' as organic carbon and export of organic carbon from the surface to the deep waters (e.g., by aggregation, mortality, and sinking of living matter and other forms of detritus). But in doing so, the model is not just a model of carbon, but rather of carbon ions. The rationale for this choice is that it is the variation in isotopic composition of carbon – formally the δ^{13} C levels – that can be measured over geological time as an indicator of the state of the oceans as well as a signature of extinctions. The main text provides an example of how variation in isotopic composition of carbonate carbon reveals both the duration and magnitude of disruptions in the carbon cycle. The take-away from analysis of the geological record is that sufficiently large, sustained perturbations are strongly associated with mass extinctions. Yet, in the case of a rapid, i.e., singular, perturbation, it may be that sufficiently large perturbations (even when not sustained) can lead to massive disruption and mass extinction.

The model describes the dynamics of the carbonate ion concentration, $c(t) \equiv [C0_3^{2-}]$, and that of the total dissolved inorganic carbon (DIC), $w(t) \equiv [CO_2]+[HCO_3^-]+[CO_3^{2-}]$. Hence, when c is high, then the water has a high capacity to neutralize acids, i.e., this is the opposite in the case of ocean acidification. The model is as follows:

$$\dot{c}/f(c) = \mu\left[1 - bs(c, c_p) - \theta\bar{s}(c, c_x) - \nu\right] + w - w_0 \tag{11}$$

$$\dot{w} = \mu\left[1 - bs(c, c_p) + \theta\bar{s}(c, c_x) + \nu\right] - w + w_0 \tag{12}$$

given non-dimensionalized time and $\mu = j_{in}\tau_w$ is a characteristic concentration. This carbon cycle model must be supplemented by a buffer function $f(c) = f_0\frac{c^\beta}{c^\beta+c_f^\beta}$, a sigmoidal equilibrium of the carbonate system $s(c, c_p) = \frac{c^\gamma}{c^\gamma+c_p^\gamma}$, and $\bar{s} = 1 - s$. The full set of parameters are as follows $\mu = 250$, $c_x = 70$, $c_p = 110$, $w_0 = 2000$, $c_f = 43.9$ all in units of μmol kg^{-1}, and the following dimensionless parameters, $b = 4$, $\theta = 8$, $\nu = 0$, $\gamma = 4$, $f_0 = 0.694$, and $\beta = 1.7$. For context, in the past 100 million years, the carbonate ion concentration has increased from $c \approx 50$ to $c > 200$, in addition a baseline DIC level is approximately 2400 μmol/kg in the oceans. These values are context for considering how deviations driven by anthropogenic input in the near term may shift the carbon cycle in the long term.

The following code represents the baseline model:

```
def ccycle(y,t,pars):
    '''
    Carbon cycle model following Rothman PNAS (2019)
    '''

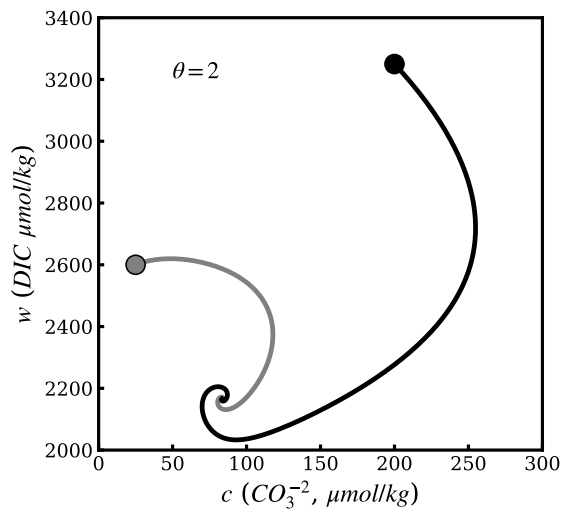
    #Variables
    c=y[0]
    w=y[1]

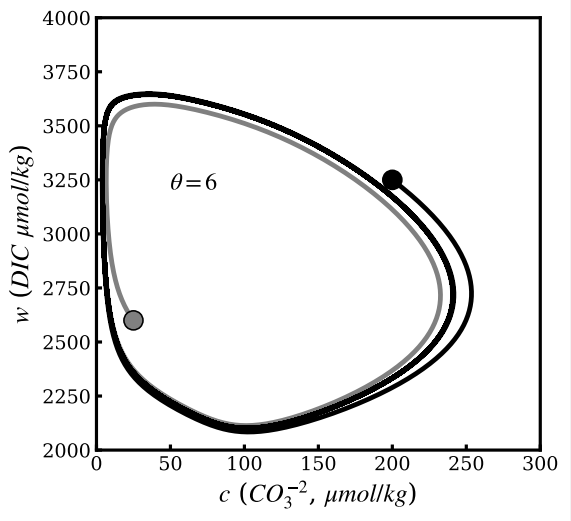
    #parameters
    sc_cp = c**pars['gamma'] / (c**pars['gamma']+pars['cp']**pars['gamma'])
    sbar_c_cx = 1-c**pars['gamma']/(c**pars['gamma']+pars['cx']**pars['gamma'])
    fc =pars['f0']*(c**pars['beta']/(c**pars['beta']+pars['cf']**pars['beta']))
```

```
#Model
dydt = np.zeros(2)
dydt[0] = fc*(pars['mu'] * \
    (1-pars['b']*sc_cp-pars['theta']*sbar_c_cx-pars['nu'])+w-pars['w0'])
dydt[1] = pars['mu'] * \
    (1-pars['b']*sc_cp+pars['theta']*sbar_c_cx+pars['nu']) - w + pars['w0']
return dydt
```

Challenge Problem: Simulating Carbon (Limit) Cycles

Simulate the model using two distinct starting conditions to demonstrate that this model can yield an attracting limit cycle, even in the absence of sustained CO_2 injection, $\nu = 0$. The following phase plane images illustrate what such dynamics should look like given two initial conditions: (i) $(c = 25, w = 2600)$; (ii) $(c = 200, w = 3250)$ in the case $\theta = 2$ (left), and (i) $(c = 25, w = 2600)$; (ii) $(c = 200, w = 3250)$ in the case $\theta = 6$ (right). As is evident, the same model can lead to convergence to equilibrium as well as stable limit cycles.





Solutions to Challenge Problem: Simulating Carbon (Limit) Cycles
The solutions here arise from a standard case of entering in parameter values and simulating a dynamical system. The following is a baseline code suitable for modification:

```
# main data goes here - units in Table S1 of Rothman
#umol/kg/yr

pars={}
pars['mu']=250
pars['cx']=70
pars['cp']=110
pars['w0']=2000
pars['cf']=43.9

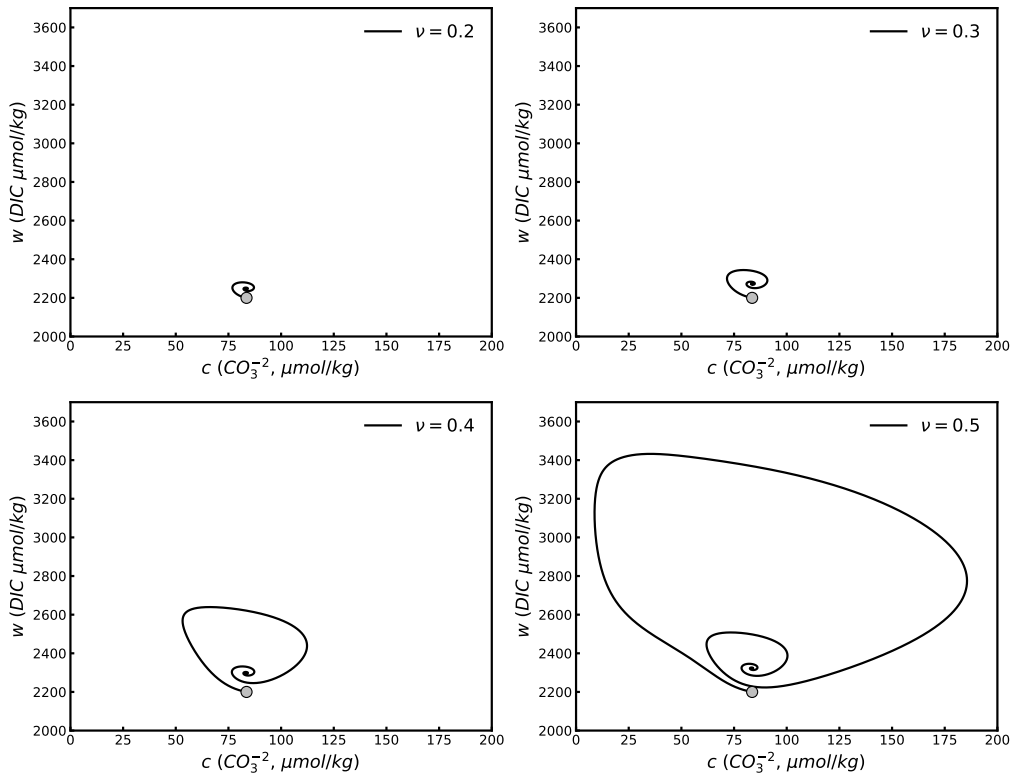
# Dimensionless
pars['b']=4
pars['theta']=2 #Modify this value
pars['nu']=0
pars['gamma']=4
pars['f0']=0.694
pars['beta']=1.70
pars['jin']=0.025
pars['tauw']=10**4

# Simulation
t=np.arange(0,100,0.001)
pars['y0']=np.array([25, 2600])
y = integrate.odeint(ccycle, pars['y0'],t,args=(pars,),rtol=10**-8)
plt.plot(y[:,0],y[:,1],c=[0.5,0.5,0.5],linewidth=3)
plt.plot(y[0,0],y[0,1],
         marker='o',
         markersize=12,
         markerfacecolor=[0.5,0.5,0.5],
         markeredgecolor='k')

pars['y0']=np.array([200, 3250])
y = integrate.odeint(ccycle,pars['y0'],t,args=(pars,),rtol=10**-8)
plt.plot(y[:,0],y[:,1],'k',linewidth=3)
plt.plot(y[0,0],y[0,1],
         marker='o',
         markersize=12,
         markerfacecolor='k',
         markeredgecolor='k')
```

B. Transition to excitability

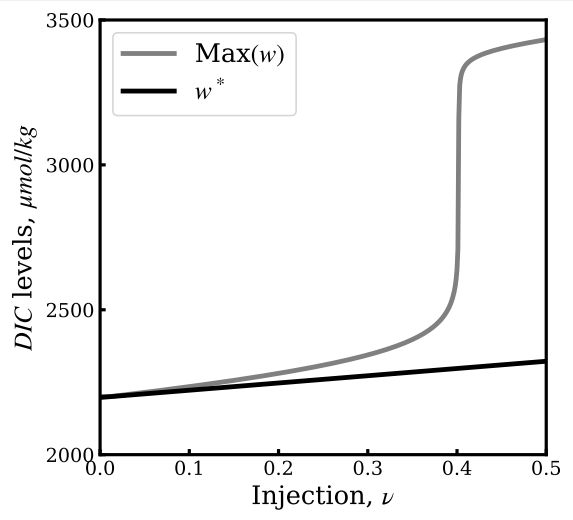
The carbon cycle model can also exhibit excitability, i.e., with different responses to small and large perturbations. The relevance of this is particularly apt in the case of injections of CO2 over short time scales, e.g., via vulcanism or anthropogenic emissions. The consequences however are radically different than many conventional perspectives. Thinking about the intrinsic dynamics of the carbon cycle makes it possible that a long-term excitatory cascade can unfold given relatively small differences in the injection. Below is an example of how changes in ν leads to increasingly large excitations despite the fact that the steady state values remain relatively unchanged. Using the same parameters as before, except changing $c_x = 55$, you should try and recapitulate these dynamics:



These dynamics also relate to the final challenge problem.

Challenge Problem: Excitations in the carbon cycle

Using the parameters in this section, contrast the changes in the fixed point as ν increases from 0 to 0.5 with the scope of the excitations. If your code works, you should find the following result, including both the excursion size and the expected state state. Compare and contrast the curves and interpret this finding.



Solutions to Challenge Problem: Excitations in the carbon cycle
Using the same parameters as above, the following code presents the change in steady state (which is relatively modest) with that maximum excursion given an initial condition of $w = 2000$ μmol/kg. The key take-away point is that although the effects of increases in ν seem modest (in terms of equilibrium), there is a dramatic change in the magnitude of the excitation, very similar to that seen in neuronal responses close to the value of $\nu = 0.4$.

```
#Steady states
cstar = (pars['b']-1)**(-1/pars['gamma'])*pars['cp']
wstar = pars['w0']+pars['mu']*(pars['theta']+pars['nu']-pars['theta']*\
            pars['cp']**pars['gamma']/((pars['b']-1)*\
                pars['cx']**pars['gamma']+pars['cp']**pars['gamma']))

#Perturbations
pars['y0']=np.array([cstar, 2200])
pars['nurange']=np.arange(0,0.501,0.001)

t=np.arange(0,100,0.001)
wmax = np.zeros(np.shape(pars['nurange']))
ystar = np.zeros((len(pars['nurange']),2))
wstar_nu = np.zeros(np.shape(pars['nurange']))
for i in range(len(pars['nurange'])):
    # Simulation compare to nu short
    pars['nu']=pars['nurange'][i]
    cstar = (pars['b']-1)**(-1/pars['gamma'])*pars['cp']
    wstar = pars['w0']+pars['mu']*(pars['theta']+pars['nu']-pars['theta']*\
            pars['cp']**pars['gamma']/((pars['b']-1)*\
                pars['cx']**pars['gamma']+pars['cp']**pars['gamma']))
    y = integrate.odeint(ccycle,pars['y0'],t,args=(pars,),rtol=10**-8)
    #wstar = y[-1,1]
    wmax[i] = np.max(y[:,1])
    ystar[i,:] = np.array([cstar,wstar])
    wstar_nu[i] = wstar

fig = plt.figure(figsize=(5,5))
ax = fig.gca()
plt.plot(pars['nurange'],wmax,c=[0.5,0.5,0.5],linewidth=3)
plt.plot(pars['nurange'],wstar_nu,'k',linewidth=3)

plt.xlabel(r'Injection, $\nu$',fontsize=20)
plt.ylabel(r'$DIC$ levels, $\mu mol/kg$',fontsize=20)
```

V. CODA

The work is done, for now. We covered a lot. Not bad for a semester. Yet there is more to do. Whether controlling the spread of infectious disease, developing evolutionarily-robust approaches to treat antibiotic resistant pathogens, improving understanding of the brain (both when it is healthy and when it is not), and characterizing the fate of interwoven ecossystems, these laboratories are a step in the direction of a transformation of the study of living systems to be immersive, infused with theory, physical intuition, and computational models. The hour is late, the time is short, all of us have much yet to do.